

Reflexión sobre el trabajo realizado

-Taller 1 Inteligencia Artificial-

Pedro Archila – 202421572

Natalia Erazo – 202213680

Jeronimo Gonzalez – 202423386

Santiago Esguerra – 202420431

Complejidad temporal y espacial:

- **DFS:**
 - **Espacial:** la complejidad espacial del peor caso corresponde a $O(n)$, siendo n la cantidad total de nodos. En este caso se debe recorrer todo el grafo para llegar del inicio a la meta, pues cada nodo solo tiene un sucesor y la meta se encuentra al final. Entonces todos los nodos se deben almacenar en la pila.
 - **Temporal:** la complejidad temporal corresponde a $O(4n)$, pues a cada nodo le corresponde máximo 4 posibles sucesores a revisar. Al simplificar se obtiene que la complejidad es $O(n)$.
- **BFS:**
 - **Espacial:** la complejidad espacial del peor caso corresponde a $O(n)$, siendo n la cantidad total de nodos. En este caso también se deben almacenar todos los nodos en la cola, ya que la meta se encuentra en último nivel del grafo.
 - **Temporal:** la complejidad temporal corresponde a $O(4n)$, pues a cada nodo le corresponde máximo 4 posibles sucesores a revisar. Al simplificar se obtiene que la complejidad es $O(n)$.
- **UCS:** La complejidad temporal del algoritmo de Dijkstra elaborado es de $O(n \log n)$ en el peor de los casos donde se tienen que visitar todos los nodos del grafo. El primer ciclo ocurre un máximo de n veces donde n representa el número de nodos y el segundo ciclo ocurre un máximo de 4 veces pues cada nodo tiene máximo 4 sucesores, uno al norte, uno al este, uno al oeste y uno

al sur, entonces la complejidad en este punto sería $4n$, pero se omiten las constantes dando como resultado $O(n)$. Además, el proceso que toma sacar y mover cosas en la priority queue tiene una complejidad de $O(\log n)$. La complejidad espacial del algoritmo es de $O(n)$ donde n es la cantidad de nodos representando el tamaño del grafo. La complejidad es 3 veces n porque tanto la priority queue como el set de nodos visitados pueden tener como máximo el número total de nodos, sin embargo, las constantes se omiten.

- **UCS con módulo obligatorio:** La complejidad temporal del algoritmo elaborado para `ModuleRepairProblem` es de $O(n \log n)$ en el peor de los casos, donde se tienen que visitar todos los estados del grafo (Siendo n la cantidad total de estados (`Posición`, `hasModule`)). El primer ciclo ocurre un máximo de n veces y el segundo ciclo ocurre un máximo de 4 veces dado que cada estado tiene un máximo de 4 sucesores (N,S,E,O), entonces la complejidad en este punto sería de $O(4n)$ pero al omitir constantes es $O(n)$. Al usar priority queue que tiene una complejidad de $O(n \log n)$ al sacar y meter objetos. La complejidad espacial del algoritmo es de $O(n)$, donde n es la cantidad de estados representando el tamaño del grafo, ya que la priority queue como el set de estados visitados pueden tener como máximo el número total de estados.
- **A*:** tiene complejidad temporal $O(b^d)$ en el peor caso, pero el factor de ramificación efectivo baja según qué tan informada sea la heurística. Se ve en los datos: en `maintenanceWeb`, con `systemRepairHeuristic` expandió 9,136 nodos en 0.1s, mientras que con `nullHeuristic` (equivalente a UCS) expandió 16,147, sobre la espacial A* debe guardar toda la frontera y el conjunto de visitados en memoria, igual que UCS, así que su espacio también es $O(b^d)$. Esto se nota en `balancedArray` (13 sistemas), donde `nullHeuristic` llegó a expandir 4.7 millones de nodos y tardó 224 segundos antes de encontrar la meta.

Complejidad de cada algoritmo en el contexto de los problemas del taller:

- **DFS:** Es completo pues siempre recorre el grafo y encuentra una solución, aunque esta no sea la menos costosa ni la más corta. Al ejecutar las pruebas siempre llega a la meta.
- **BFS:** Es completo pues siempre recorre el grafo y encuentra la solución con menor número de movimientos, aunque esta no sea la menos costosa. Al ejecutar las pruebas siempre llega a la meta.
- **UCS:** Tras probar todos los mapas posibles para el algoritmo de UCS se puede evidenciar que siempre encuentra un camino hasta la meta de manera muy rápida. En todos los casos, el robot puede llegar hasta la meta D utilizando el camino más corto.
- **UCS con módulo obligatorio:** Es completo porque usa el mismo algoritmo de UCS sobre un espacio de estados finito (Posicion,hasModule), con un conjunto de visitados que garantiza que se agoten los estados alcanzables si es necesario. Al probar los 10 mapas disponibles, el robot siempre llegó al centro de control C habiendo recogido el módulo M primero, sin fallar ni quedarse colgado en ningún caso.
- **A*:** es completo en los problemas del taller porque el espacio de estados es finito y mi implementación usa un conjunto de visitados que evita reexplorar el mismo estado, evitando ciclos infinitos.

Condiciones del algoritmo para encontrar una solución óptima:

- **DFS:** Solo sería óptimo cuando todas las soluciones tienen la misma profundidad. No garantiza encontrar una solución óptima, ya que prioriza recorrer los nodos más profundos sin tener en cuenta la cantidad de pasos

ni el costo del camino. Puede encontrar una solución antes de encontrar otra que tenga menos pasos.

- **BFS:** Encuentra una solución óptima en cuanto al número de pasos cuando todas las acciones tienen el mismo costo. Esto se debe a que recorre los nodos por niveles, por lo que la primera solución que encuentra corresponde al camino con menor cantidad de pasos. Si las acciones tienen costos diferentes, BFS no garantiza la solución de menor costo.
- **UCS:** Las condiciones necesarias para el funcionamiento del algoritmo de Dijkstra son saber dónde empieza, en qué momento se visita la meta y los costos de cada movimiento hacia alguno de sus 4 sucesores. Este algoritmo es bastante simple en cuanto a las condiciones para encontrar la mejor solución porque este es un algoritmo que por su funcionamiento siempre encuentra el camino más óptimo desde un punto específico a otro.
- **UCS con módulo obligatorio:**
Las condiciones necesarias son similares a las de USC, saber el estado inicial, la meta y el costo de cada movimiento, solo que aquí el estado también dice si ya se tiene el módulo. Como el costo nunca es cero ni negativo, siempre encuentra la ruta más barata; lo confirmé comparando, en los 10 mapas de módulo, el costo que reporta la búsqueda contra el costo real del juego — siempre coincidieron.
- **A*:** encuentra la solución óptima si la heurística es admisible (nunca sobreestima el costo real restante); en búsqueda en grafo con visitados, además necesita ser consistente para no tener que reabrir nodos. Mis tres heurísticas cumplen esto, y se confirma porque el costo final fue idéntico (24, 50, 68, 108, 117, 181, 201...) sin importar cuál se usó.

¿Por qué la posición del robot no es suficiente para representar el problema en el ejercicio 4 y porque *hasModule* debe hacer parte del estado?

Si el estado fuera solo la posición (x, y), el algoritmo vería la celda donde está M como el mismo estado sin importar si el robot ya lo recogió o no. Eso rompe dos

cosas: la prueba de meta no podría distinguir "llegó a C sin M" de "llegó a C con M", y una celda ya visitada sin el módulo quedaría marcada como definitiva, sin poder volver a considerarla ya con el módulo (que cuesta el doble). Por eso hasModule tiene que ir en el estado: así cada combinación posición + condición es un nodo distinto del grafo, y UCS puede comparar correctamente todas las rutas posibles a M y de ahí a C.

Consistencia de las heurísticas:

- ***manhattanHeuristic:***

Es consistente: calcula la distancia Manhattan hasta el objetivo de la fase actual (K, el T más cercano, o C), y esa distancia cambia máximo 1 por cada movimiento. Como cada paso cuesta 1, nunca sobreestima el costo real restante.

- ***euclideanHeuristic:***

Es consistente por el mismo argumento que Manhattan (la distancia cambia máximo 1 por paso). Al no considerar los obstáculos del mapa, casi siempre subestima más que Manhattan, por lo que es admisible pero menos informada.

- **Heurística propia:**

Toma el máximo de la distancia a cada objetivo pendiente (kit, sistemas por reparar, y C), y es admisible porque el robot tiene que llegar al menos al más lejano de esos puntos. En la práctica expandió menos nodos que las otras tres heurísticas en todos los mapas que probé.

Comparación entre todas las heurísticas en términos de costo, nodos expandidos y tiempo de ejecución

Mapa (# de T)	Heurística	Costo	Nodos expandidos	Tiempo (s)

tinyRepair (1)	null	24	148	0.0005
tinyRepair (1)	manhattan	24	127	0.0005
tinyRepair (1)	euclidean	24	129	0.0006
tinyRepair (1)	propia	24	115	0.0005
controlWing (3)	null	69	781	0.0027
controlWing (3)	manhattan	69	668	0.0024
controlWing (3)	euclidean	69	678	0.0030
controlWing (3)	propia	69	631	0.0023
repairLab (5)	null	108	6385	0.0191
repairLab (5)	manhattan	108	5978	0.0221
repairLab (5)	euclidean	108	6089	0.0306
repairLab (5)	propia	108	5709	0.0224
sixBayDeck (6)	null	114	14473	0.0476
sixBayDeck (6)	propia	114	10698	0.0448
sensorArray (8)	null	135	80522	0.3146
sensorArray (8)	manhattan	135	76452	0.3660
sensorArray (8)	euclidean	135	77501	0.5389
sensorArray (8)	propia	135	66143	0.3340
nineBlockGrid (9)	null	162	190992	0.8334
nineBlockGrid (9)	manhattan	162	185546	0.9987
nineBlockGrid (9)	propia	162	161858	0.8936
elevenRoomDeck (11)	null	201	953811	5.66
elevenRoomDeck (11)	propia	201	923891	6.70

El costo final fue idéntico entre las 4 heurísticas en todos los mapas probados, lo que confirma que las tres son admisibles. La heurística propia expandió consistentemente menos nodos que nullHeuristic, manhattanHeuristic y euclideanHeuristic), y el crecimiento de nodos expandidos frente al número de sistemas T es claramente exponencial (de aprox 148 nodos con 1 sistema a aproximadamente 950.000 con 11). Los tiempos tomados fueron tomados una única vez

Reflexión sobre el uso de inteligencia artificial

La inteligencia artificial se utilizó para entender errores en consola y localizar la raíz del problema, también para verificar que el código cumpliera con los requerimientos (encontrar el camino con menos pasos en BFS). En este caso se utilizó Copilot en VScode Para la parte de las complejidades se utilizó Gemini y ChatGPT para verificar que fueran acertadas.

Sintaxis manhattan:

Se aceptaron los cambios pues todos eran relevantes y correspondían a sintaxis. La lógica propuesta sigue siendo la misma. El funcionamiento se verificó desde la interfaz del programa, pues ya no salía el error y la heurística era correcta.

```
def manhattanHeuristic(state, problem):  
    """  
    The Manhattan distance heuristic.  
  
    Baseline rule for this workshop: estimate the direct distance to the next  
    mandatory target:  
    - K if the robot does not have the kit yet.  
    - the nearest pending T if the robot has the kit and systems remain.  
    - C if all systems have been repaired.  
    """  
    position, hasKit, pendingSystems = state  
  
    if not hasKit:  
        target = problem.kitPosition  
    elif len(pendingSystems) > 0:  
        target = min(pendingSystems,  
                      key=lambda t: abs(position[0] - t[0]) + abs(position[1] - t[1]))  
    else:  
        target = problem.controlPosition  
    return abs(position[0] - target[0]) + abs(position[1] - target[1])
```

```
def manhattanHeuristic(state, problem):
    """
    The Manhattan distance heuristic.

    Baseline rule for this workshop: estimate the direct distance to the next
    mandatory target:
    - K if the robot does not have the kit yet.
    - the nearest pending T if the robot has the kit and systems remain.
    - C if all systems have been repaired.
    """
    position, hasKit, pendingSystems = state

    if not hasKit:
        target = problem.kitPosition
    elif len(pendingSystems) > 0:
        target = min(
            pendingSystems,
            key=lambda t: abs(position[0] - t[0]) + abs(position[1] - t[1]),
        )
    else:
        target = problem.controlPosition

    return abs(position[0] - target[0]) + abs(position[1] - target[1])
```

Prompts:

mira esto me salio: iaArtificial/EstacionEspacial \$ python menu.py

Ejecutando:

```
/Users/nataliaerazo/.pyenv/versions/3.9.18/bin/python
"/Users/nataliaerazo/Desktop/Universidad/Inteligencia
artificial/Taller1_inteligenciaArtificial/EstacionEspacial/main.py" -p DiagnosticProblem -f
depthFirstSearch -l tinyDiagnostic -x 0.25 -z 1.0
```

RobotPositions: [(2, 10)]

MissionSymbols: {'D': [(18, 2)], 'C': [], 'M': [], 'K': [], 'T': []}

Traceback (most recent call last):

```
File "/Users/nataliaerazo/Desktop/Universidad/Inteligencia
artificial/Taller1_inteligenciaArtificial/EstacionEspacial/main.py", line 284, in <module>
args = readCommand(sys.argv[1:])
File "/Users/nataliaerazo/Desktop/Universidad/Inteligencia
artificial/Taller1_inteligenciaArtificial/EstacionEspacial/main.py", line 144, in readCommand
robotAgentType = loadAgent("SearchAgent")
File "/Users/nataliaerazo/Desktop/Universidad/Inteligencia
artificial/Taller1_inteligenciaArtificial/EstacionEspacial/main.py", line 198, in loadAgent
module = import("algorithms.agents", fromlist=[agentName])
File "/Users/nataliaerazo/Desktop/Universidad/Inteligencia
artificial/Taller1_inteligenciaArtificial/EstacionEspacial/algorithms/agents.py", line 3, in
<module>
import algorithms.search as search
File "/Users/nataliaerazo/Desktop/Universidad/Inteligencia
artificial/Taller1_inteligenciaArtificial/EstacionEspacial/algorithms/search.py", line 4, in
<module>
from algorithms.heuristics import nullHeuristic
File "/Users/nataliaerazo/Desktop/Universidad/Inteligencia
artificial/Taller1_inteligenciaArtificial/EstacionEspacial/algorithms/heuristics.py", line 29
target min( pendingSystems,
^
SyntaxError: invalid syntax
```

La ejecucion termino con codigo 1.

Presiona Enter para volver al menu...

Optimized tool selection

Voy a revisar la función que está causando el error en el heurístico y corregirlo, luego lo probaré ejecutando el mismo comando para verificar que quede resuelto.

Reviewed 2 files

Ya confirmé la causa: hay un error de sintaxis directo en algorithms/heuristics.py, y además revisé que algorithms/search.py tiene varios fallos de API que también pueden romper la ejecución. Voy a corregir ambos con el menor cambio posible y volver a ejecutar la misma prueba.

Reviewed 4 files and searched for regex patterns

Camino con menos pasos BFS:

Se modificó el lugar donde se hace el push de inicio y el cambio se realizó según lo indicado porque modificaba sutilmente el orden de los pasos. El funcionamiento se verificó desde la interfaz del programa, pues al seleccionar el Modo de animación como Modo texto ahora sí era evidente que el personaje está siguiendo un camino con menos pasos.

```
def breadthFirstSearch(problem: SearchProblem):  
    """  
    Search the shallowest nodes in the search tree first.  
    """  
    BFS must minimize the number of actions (moves), not the terrain cost.  
    The frontier is FIFO, and a state is marked visited when it is enqueued,  
    ensuring that the first time a state is discovered corresponds to the  
    shortest path to it in an unweighted graph.  
    """  
    # TODO: Add your code here  
    inicio = problem.getStartState()  
    visitado = set()  
    cola = Queue()  
  
    visitado.add(inicio)  
  
    cola.push((inicio, []))  
  
    while not cola.isEmpty():  
        nodo, movimientos = cola.pop()  
  
        if problem.isGoalState(nodo):  
            return movimientos  
  
        for sucesor, accion, costo in problem.getSuccessors(nodo):  
            if sucesor not in visitado:  
                visitado.add(sucesor)  
                cola.push((sucesor, movimientos + [accion]))  
  
    return None
```

Prompts:

mira tengo este para bfs, pero creo q no cumple con lo de menor cantidad de movimientos:

```
def breadthFirstSearch(problem: SearchProblem):  
    """  
    Search the shallowest nodes in the search tree first.  
    """  
    visitado = set()  
    cola = Queue()  
    cola.push((problem.getStartState(), []))  
  
    while not cola.isEmpty():  
        nodo, movimientos = cola.pop()  
  
        if problem.isGoalState(nodo):  
            return movimientos  
  
        if nodo not in visitado:  
            visitado.add(nodo)  
            for sucesor, accion, costo in problem.getSuccessors(nodo):  
                if sucesor not in visitado:  
                    cola.push((sucesor, movimientos + [accion]))  
  
    return None
```

Tu intuición es correcta: tal como está escrito, este BFS **no garantiza** encontrar la ruta con la menor cantidad de movimientos en todos los casos ni es eficiente. [PDF](#)

El problema está en **cuándo marcas los nodos como visitados**. En tu código actual, marcas el nodo como visitado al momento de sacarlo de la cola (`pop`). Si un mismo nodo es alcanzado por dos caminos distintos antes de salir de la cola, se inserta múltiples veces, lo que genera exploraciones redundantes e innecesarias.

Para arreglarlo y asegurar la solución óptima en número de pasos (como pide el Punto 2), debes registrar el nodo en `visitado` en el momento exacto en que lo insertas en la cola (`push`).

[PDF +1](#)

Código corregido para BFS

```
Python  
from utils import Queue  
  
def breadthFirstSearch(problem):  
    """  
    Explora los nodos más superficiales primero para garantizar  
    el menor número de movimientos.  
    """  
    estado_inicial = problem.getStartState()  
    if problem.isGoalState(estado_inicial):  
        return []
```